Part 2: Tour of R Functions

Summary

In this Part we take a whirlwind tour through some of the most used functions in R.

Statistical analysis procedures are omitted at this point. Also, functions generally used in writing code (e.g., `while()`, etc.), along with graphics/visualization functions, are left to other Parts.

Instead, focus is placed on functions for manipulating vectors and matrices and performing crucial mathematical calculations.

Getting Going, and Quitting

After starting R, ensure you are in the desired working directory. R will write and read files, and otherwise store data in this directory. The working directory is set using the `setwd()` function:

```
> setwd("/Users/chadschafer")
> getwd()
[1] "/Users/chadschafer"
```

To quit R, use `q()`:

```
> q()
```

You will be asked if you want to save the current objects. If you do so, R will create a (hidden) file in the current working directory that will automatically be reloaded when you return to this working directory.

Argument Matching

As seen previously, the `help()` function provides detailed argument lists for each function. The `args()` function can also be used:

```
> args(sample)
function (x, size, replace = FALSE, prob = NULL)
NULL
```

Here we see that there are four arguments to `sample()`. Two of them have default values: `replace = FALSE` and `prob = NULL`.

When calling `sample()`, one can either provide arguments with names, in any order, or without names, in the order as they are given in the `args(sample)` output.

Also, only enough of the argument name need be provided that uniquely specifies the argument.

In other words, these commands will all work equally well:

```
> sample(1:100, 10, FALSE)
> sample(1:100, size=10, replace=FALSE)
> sample(x=1:100, rep=F, si=10)
> sample(1:100, 10, r=F)
```

Basic Math Functions

Each of the following does exactly what you would expect when passed a numeric (integer or double) vector `x`. As usual, see `help()` for arguments that expand/alter their capabilities.

```
> sum(x)
> max(x)
> min(x)
> sort(x)
> rank(x)
> mean(x)
> var(x)
> sd(x)
```


Handling Missing Values

Most of the functions on the previous page have arguments that control how `NA` values should be handled. Often, but not always, this argument is named `na.rm` (to stand for "NA Remove") and by default it is set to `FALSE`, meaning that `NA` values in the vector will result in `NA` output.

```
> max(c(45, 78, 12, NA))
[1] NA
```

```
> max(c(45, 78, 12, NA), na.rm = TRUE)
[1] 78
```

`sort()` and `rank()` are special cases. By default, `NA` values are removed, but see argument `na.last`.

Mathematical functions that take scalar inputs will return a vector when passed a vector:

```
> sqrt(c(1,2,3))
[1] 1.000000 1.414214 1.732051
```

```
> log(x) # Default is natural log
> exp(x)
> abs(x)
> tan(x)
> round(x)
> floor(x)
> ceiling(x)
> factorial(x)
```


Vector Arithmetic

Vectors and matrices of the same size can be added and subtracted, resulting in another object of the same size.

```
> c(4,5,6) - c(3,2,1)
[1] 1 3 5
```

Multiplication using $*$ is element-by-element, **even with matrices**:

```
> matrix(c(1,2,3,4),nrow=2) * matrix(c(1,2,3,4),nrow=2)
     [,1] [,2]
[1,]    1    9
[2,]    4   16
```

The operator %% finds the "modulus."

Exercise: What happens when one tries to add/subtract/multiply vectors of differing lengths?

Logical Operators

The operators `|` and `&` perform elementwise logical OR and logical AND comparisons, respectively:

```
> c(T,T,F) | c(T,F,F)
[1]  TRUE  TRUE FALSE
```

```
> c(T,T,F) & c(T,F,F)
[1]  TRUE FALSE FALSE
```

The `!` operator negates:

```
> !(c(T,T,F) | c(T,F,F))
[1] FALSE FALSE  TRUE
```

The functions `any()`, `all()`, and `which()` take logical vectors as input:

Are **any** of the simulated values greater than 4?

```
> any(rnorm(1000)>4)
[1] FALSE
```

Are **all** of the simulated values less than 3?

```
> all(rnorm(1000)<3)
[1] FALSE
```

Which entries have values larger than 2.5?

```
> which(rnorm(1000)>2.5)
[1]  73 185 197 281 541 791 888
```


Set Functions

There are functions that treat vectors like sets:

```
> intersect(c(1,2,3),c(2,3,4,5))
[1] 2 3
```

```
> union(c(1,2,3),c(2,3,4,5))
[1] 1 2 3 4 5
```

```
> setdiff(c(1,2,3),c(2,3,4,5))
[1] 1
```

```
> setequal(c(1,2,3),c(3,2,1))
[1] TRUE
```

Exercise: Note that `setdiff()` is not symmetric. Write a line of R code that creates a symmetric difference between sets `x` and `y`.

Finding an Element

Operator `%in%` tests if a value is in a vector or matrix, and `match()` returns the **first** position of that value in a vector.

```
> "GOOG" %in% c("GOOG", "SBUX", "AAPL")
[1] TRUE
```

```
> match("SBUX", c("GOOG", "SBUX", "AAPL"))
[1] 2
```

```
> match(4,c(4,1,7,4,2,4))
[1] 1
```

```
> which(c(4,1,7,4,2,4) == 4)
[1] 1 4 6
```

Matrix Algebra

R has great capacity to do matrix algebra, albeit not at the same speed as Matlab. The operator to perform matrix multiplication is `%*%`. Other useful functions (assuming `A` is a matrix of form appropriate for the function):

```
> t(A)      #  Matrix Transpose
> nrow(A)
> ncol(A)
> chol(A)
> eigen(A)
> svd(A)
> qr(A)
> diag(A)   # Extract the diagonal
> solve(A)  # Matrix Inverse
```


Exercise: Look at `help(diag)` and explore the various ways this function gets utilized.

Working with Named Distributions

Each standard, “named" distribution has a suite of four useful functions for doing basic probability calculations.

For example, for the normal distribution:

Generate a (pseudo-)random sample of size 5 from the normal distribution with mean 20 and standard deviation 3:

```
> set.seed(0)
> rnorm(5, 20, 3)
[1] 23.78886 19.02130 23.98940 23.81729 21.24392
```

The function `set.seed()` controls the seed of the random number generator in R.

Evaluate the CDF: If X has the normal distribution with mean 10 and standard deviation 2, what is $P(X \leq 10.5)$?

```
> pnorm(10.5, 10, 2)
[1] 0.5987063
```

Find Quantiles: If Y has the normal distribution with mean 5 and standard deviation 3, what value of c is such that $P(Y \leq c) = 0.95$?

```
> qnorm(0.95, 5, 3)
[1] 9.934561
```

Evaluate the PDF: Evaluate the standard normal density at 0.5.

```
> dnorm(0.5) #  Defaults are mean=0, sd=1
[1] 0.3520653
```

Such functions exist for all of the common distributions:

Exponential: `rexp(),pexp(),qexp(),dexp()`

Gamma: `rgamma(),pgamma(),qgamma(),dgamma()`

Poisson: `rpois(),ppois(),qpois(),dpois()`

Binomial: `rbinom(),pbinom(),qbinom(),dbinom()`

Chi-Square: `rchisq(),pchisq(),qchisq(),dchisq()`

t: `rt(),pt(),qt(),dt()`

Uniform: `runif(),punif(),qunif(),dunif()`

Carefully inspect the arguments: There are different ways of parameterizing a distribution.

Exercise: Write code, without using `rnorm()`, to generate random variables from the normal distribution.

Assigning Values to Objects

In some situations it can be very useful to dynamically construct and reference object names. The `assign()` and `get()` functions enable this.

```
> objname = "x"
> assign(objname, 5)
> print(x)
[1] 5
```

```
> get(objname)
[1] 5
```


The `do.call()` function extends this idea to calling functions. Here, the function to be called is provided first, followed by the arguments to that function as a list:

```
> do.call("mean", list(1:10, trim = 0.1))
[1] 5.5
```

Sampling a Vector

The `sample()` function gets a great deal of use, e.g. when generating a random subsample of a data set, when running a bootstrap, when randomly permuting a list, etc.

One specifies the vector from which to sample, the size of the sample to draw, whether sampling is with or without replacement:

```
> sample(1:10, 5, FALSE)
[1]  9  3 10  5  6
```

```
> sample(1:10, 5, TRUE)
[1]  3  9 10  7  7
```


Exercise: In K-Fold Cross Validation a data set is divided into `K` distinct groups, of size as close to equal as possible. Analysis then proceeds with one fold omitted in order to test performance. Let `n` denote the sample size. Construct an object `fold` which holds the fold assignment, determined at random.

Apply Functions

R has a family of apply functions that allow the user to efficiently run another function on subsets of another object. The simplest example is the use of `apply()` on a matrix. Here, `matex` is a 2 by 5 matrix:

```
> matex = matrix(1:10, nrow=2)
```

Find the sum of each row, and the mean of each column:

```
> apply(matex,1,sum)
[1] 25 30
```

```
> apply(matex,2,mean)
[1] 1.5 3.5 5.5 7.5 9.5
```

This function can also be used on an array.

The `sweep()` function is often used in combination with `apply()`. It allows the user to perform an operation on every row or column of a matrix. The result is a matrx of the same dimension of the original matrix.

The following example "centers" each column of `matex` on zero:

```
> sweep(matex, MAR = 2, apply(matex,2,mean), FUN="-")
     [,1] [,2] [,3] [,4] [,5]
[1,] -0.5 -0.5 -0.5 -0.5 -0.5
[2,]  0.5  0.5  0.5  0.5  0.5
```

Here, `FUN` specifies the operation to be applied on each column (since `MAR = 2`). The amount to be subtracted is each column mean, as calculated by `apply(matex,2,mean)`.

There are additional apply functions to be used on more complex objects. For example, the function `lapply()` works on the components of a list:

```
> listex = list(c(T,F,T), 1:5, c("a","b","c","d"))
> lapply(listex, typeof)
[[1]]
[1] "logical"

[[2]]
[1] "integer"

[[3]]
[1] "character"
```


Exercise: Take a look at `help(sapply)`. How does this function differ from `lapply()`?

Split

The `split()` function breaks an object into a list using a specified grouping variable. First, a simple example:

```
> split(1:5, c(1,1,2,2,2))
$`1`
[1] 1 2

$`2`
[1] 3 4 5
```


In our "gradebook" example, `split()` can group the rows:

```
> split(gradebook, gradebook$FinalGrade)
$A
    Name HWScore ExamScore FinalGrade
1 Miguel      98        92          A
3   Mary      99        87          A

$`A-`
   Name HWScore ExamScore FinalGrade
2 Yotam      87        91         A-

$B
  Name HWScore ExamScore FinalGrade
4 Jing      78        95          B
```

Exercise: How could I find the average homework score for students of each final letter grade?

Working with Strings

R has an range of functions for working with strings. The most commonly used includes `paste()`, for concatenating strings. Coercion of non-string types is done automatically.

```
> paste("abc","def")
[1] "abc def"
```

```
> paste("abc","def",sep="-")
[1] "abc-def"
```

```
> paste0("abc",30)
[1] "abc30"
```

Note that `paste0` uses `sep=""` by default.

Substrings can be extracted or assigned using `substr()`:

```
> strex = "abcdef"
> substr(strex,2,4)
[1] "bcd"
```

```
> substr(strex,3,5) = "qed"
> print(strex)
[1] "abqedf"
```

Note that this syntax does **not** find the substring:

```
> strex[2:4]
[1] NA NA NA
```


Exercise: Explain the behavior seen in the previous example. Also, look at `length(strex)` and `nchar(strex)` for a related issue.

The function `strsplit()` breaks a string into components based on a provided "split" string:

```
> strsplit("file_00001_A", "_")
[[1]]
[1] "file"  "00001" "A"
```

A vector of strings becomes a list:

```
> strsplit(c("file_00001_A", "file_00002_A"), "_")
[[1]]
[1] "file"  "00001" "A"

[[2]]
[1] "file"  "00002" "A"
```


Exercise: Suppose that object `fnamelist` is a vector consisting of filenames, each is of the form `A_B_C`, where `B` is a number with leading zeros to force it to be of length 5. Your objective is to extract the numbers from this center section, in numeric form. How would you do this (1) if the `A` portion is always the same length, (2) if the length of the `A` portion varies from file to file?

Installing Packages

The development of R is largely a decentralized effort, with users working to extend its functionality with add-on packages built following a stanard format to enable others to easily install and utilize these new functions.

The CRAN website https://cran.r-project.org/web/packages/ has a list of packages that have been posted to their repository.

The range of packages available is staggering, with almost any functionality you could imagine have been implemented by someone. Quality can vary, however, and there can be inconsistency in how R syntax is utilized.

Packages can be installed from the command line. For example, package `tidyverse` is very useful, enabling a wide range of advanced data manipulation and visualization tools that we will use later on

```
> install.packages("tidyverse")
```

A package only needs to be installed once, but it needs to loaded into each R session where it is to be used. This is achieved with the `library()` function:

```
> library(tidyverse)
```

Package `dplyr`

Some of the capabilities of package `dplyr`, which is installed as part of `tidyverse`, are worth pointing out now.

The `filter()` function allows one to easily extract particular rows of a data frame based on one or multiple conditions:

```
> filter(gradebook, HWScore > 90 & ExamScore > 90)
    Name HWScore ExamScore FinalGrade
1 Miguel      98        92          A
```

```
> filter(gradebook, ExamScore >= 95 | FinalGrade == "A")
    Name HWScore ExamScore FinalGrade
1 Miguel      98        92          A
2   Mary      99        87          A
3   Jing      78        95          B
```


The `arrange()` function will reorder the rows of a data frame using the specified column(s). The `desc()` function switches to descending order.

```
> arrange(gradebook,HWScore)
    Name HWScore ExamScore FinalGrade
1   Jing      78        95          B
2  Yotam      87        91         A-
3 Miguel      98        92          A
4   Mary      99        87          A
```

```
> arrange(gradebook,desc(HWScore))
    Name HWScore ExamScore FinalGrade
1   Mary      99        87          A
2 Miguel      98        92          A
3  Yotam      87        91         A-
4   Jing      78        95          B
```

Multiple columns can be provided to break ties.

The `select()` function extracts particular variables from a data frame.

```
> select(gradebook, Name, FinalGrade)
Warning: `lang()` is deprecated as of rlang 0.2.0.
Please use `call2()` instead.
This warning is displayed once per session.
Warning: `new_overscope()` is deprecated as of rlang 0.2.0.
Please use `new_data_mask()` instead.
This warning is displayed once per session.
Warning: `overscope_eval_next()` is deprecated as of rlang 0.2.0.
Please use `eval_tidy()` with a data mask instead.
This warning is displayed once per session.
    Name FinalGrade
1 Miguel          A
2  Yotam         A-
3   Mary          A
4   Jing          B
```


Look at `help(select_helpers)` for functions that can make things even easier, e.g.,

```
> select(gradebook, ends_with("Score"))
  HWScore ExamScore
1      98        92
2      87        91
3      99        87
4      78        95
```